(CE7456 Lecture Notes 7) Deferred Data Structuring (a.k.a. Database Cracking and Merging)

Yufei Tao

Traditional indexing first creates an index on the whole dataset before starting to answer queries. For example, after building a binary search tree (BST) on an (unsorted) set S of n integers in $O(n \log n)$ time, we can use the tree to answer each predecessor query[1] in $O(\log n)$ time. This paradigm, however, falls short when the dataset S will only be searched a small number of times. In the extreme, if only one query needs to be answered, the best approach is to scan S in full, which requires only $O(n)$ time.

Deferred data structuring (DDS) asks the question: if we are to answer r queries in an *online* manner — that is, the next query is given after the result of the previous query has been output — what is the smallest total execution time (of r queries)? This question arises in many applications where large volumes of data are collected but only sparingly queried. In those environments, the problem size n is huge such that even sorting is considered expensive and should be avoided as much as possible. Furthermore, it is impossible to predict how many queries — namely, the integer r mentioned earlier — will need to be supported. Instead of constructing a complete index right away, a better algorithm is to carry out only a calculated portion of the construction during each query. If r eventually reaches a certain threshold, the index will be created in full, but it is more likely that r will stop at some value significantly lower than that threshold. The challenge is to ensure "smoothness": as r grows, the total cost of all the r queries *so far* ought to increase as slowly as possible.

For a large class of DDS problems (such as predecessor search), it is known [2] that the total time of answering r queries from a non-preprocessed dataset must be $\Omega(n \log(1 + r))$ [2] for <u>*every*</u> value of $r \in [1, n]$. In today's lecture, we will discuss techniques for matching this lower bound.

Math Conventions. For any integer $x \geq 1$, let $[x]$ represent the set $\{1, 2, ..., x\}$. Every logarithm has base 2 by default. Define $\text{Log}(x) = \log(1 + x)$ for any $x \geq 1$.

1 Formalization of DDS

Let $\mathbb{D}$ be a (possibly infinite) set called the *data domain*, whose members are called *data elements*. Let $\mathbb{Q}$ be a (possibly infinite) set called the *query domain*, whose members are called *queries*. Over a finite $S \subseteq \mathbb{D}$, each query $q \in \mathbb{Q}$ returns an *answer*, represented as $\textsc{Ans}_q(S)$. In a (data structure) *problem* characterized by $(\mathbb{D}, \mathbb{Q})$, the goal is to store a given set $S \subseteq \mathbb{D}$ — where S is called the *dataset* — in a data structure that allows us to compute $\textsc{Ans}_q(S)$ efficiently for any $q \in \mathbb{Q}$.

We consider that, for any $q \in \mathbb{Q}$, the answer $\textsc{Ans}_q(S)$ can be represented using $O(1)$ words and can be computed in $O(n)$ time (which essentially means that the query can be answered using a brute-force algorithm such as exhaustive scan).

We now formalize the DDS version of the problem parameterized by $(\mathbb{D}, \mathbb{Q})$. Initially, an algorithm $\mathcal{A}$ is given the dataset S in an array, where the elements are arbitrarily permuted. An

[1] Given a search value q, a predecessor query returns the largest element in S that does not exceed q.

adversary first chooses a query $q_1 \in \mathbb{Q}$ and solicits the answer $\textsc{Ans}_{q_1}(S)$ from $\mathcal{A}$. Iteratively, for each $i \in [2, n]$, after the answer of the $(i-1)$-th query has been obtained, the adversary chooses the next query $q_i \in \mathbb{Q}$ and solicits $\textsc{Ans}_{q_i}(S)$ from $\mathcal{A}$. The adversary is permitted to observe the execution of $\mathcal{A}$ and, thus, capable of selecting a "bad" q_i for $\mathcal{A}$.

The algorithm $\mathcal{A}$ is said to guarantee *running time* $\textsc{Time}(n, r)$ if, for *every* $r \in [1, n]$, the first r queries are processed with a total cost at most $\textsc{Time}(n, r)$.

2 The Class of Spectrum-Indexable Problems

A problem parameterized by $(\mathbb{D}, \mathbb{Q})$ is $(B(n), Q(n))$ *spectrum indexable* if the following property holds on every dataset $S \subseteq \mathbb{D}$:

> For every integer $s \in [|S|]$, it is possible to construct a data structure on S in $O(|S| \cdot B(s))$ time that can answer any query in $O(\frac{|S|}{s} \cdot Q(s))$ time.

The term "spectrum indexable" is chosen to reflect the ability to build a "good" index structure — as far as functions $B(n)$ and $Q(n)$ are concerned — for the whole spectrum of the parameter s.

Two observations are in order here:

- $(B(n), Q(n))$ *spectrum indexability* implies that we can build a data structure on any dataset $S \subseteq \mathbb{D}$ in $O(|S| \cdot B(|S|))$ time to answer a query in $O(Q(|S|))$ time (for this purpose, simply set $s = |S|$).
- Consider any decomposable problem with the following property: for any dataset $S \subseteq \mathbb{D}$, we can build a data structure $\mathcal{T}$ in $O(|S| \cdot B(|S|))$ time to answer a query in $O(Q(|S|))$ time. Then, the problem must be $(B(n), Q(n))$ spectrum indexable. To see why, given an integer $s \in [|S|]$, divide S arbitrarily into $m = \lceil |S|/s \rceil$ disjoint subsets $S_1, S_2, ..., S_m$ where all subsets have size s except S_m. For each $i \in [m]$, create a structure $\mathcal{T}(S_i)$ in $O(|S_i| \cdot B(s))$ time; the total time to create all the m structures is $O(m \cdot s \cdot B(s)) = O(|S| \cdot B(s))$. To answer a query q, simply search every $\mathcal{T}(S_i)$ to obtain $\textsc{Ans}_q(S_i)$ in $O(Q(s)))$ time and then combine $\textsc{Ans}_q(S_1), \textsc{Ans}_q(S_2), ..., \textsc{Ans}_q(S_m)$ into $\textsc{Ans}_q(S)$ using $O(m)$ time. The total query time is therefore $O(m \cdot Q(s))$.

3 Merging and Cracking

This section will introduce two popular techniques — named *merging* and *cracking* — for tackling DDS problems, using predecessor search as a representative example. In this problem, S is a set of n integers. Each query $q \in \mathbb{Q}$ is an integer; and $\textsc{Ans}_q(S)$ is the largest integer in S at most q, or `null` if all the integers in S are greater than q. The problem is $(O(\log n), O(\log n))$ spectrum indexable (think: why?).

Next, we will give two proofs — each based on a different technique — that the DDS version of the predecessor search problem can be solved in $\textsc{Time}(n, r) = O(n \operatorname{Log} r)$ time.

Merging. We assume, w.l.o.g., that n is a power of 2. Let us first describe an algorithm that works under the condition $r \operatorname{Log} r \leq n/2$.

At all times, we partition S arbitrarily into disjoint subsets — referred to as *runs* — each having the same size $s = 2^i$ for some $i \geq 0$. Every run is sorted and stored in an array. Initially, $s = 1$,

i.e., a run contains one individual element of S. Over time, the run size s increases monotonically. Whenever s needs to go from 2^i to 2^j for some value $j > i$, an *overhaul* is carried out to build the new runs. As a run of size 2^j can be obtained by merging 2^{j-i} runs of size 2^i in $O(2^j \cdot (j-i))$ time, the overhaul can be completed in $O(n \cdot (j-i))$ time. Therefore, if the current run size is s, the overall cost of producing all the runs in history is $O(n \operatorname{Log} s)$.

A (predecessor) query is answered simply by performing binary search on every run. To control the cost, however, the algorithm makes sure that the current size s is at least $i \operatorname{Log} i$ before processing the i-th ($i \geq 1$) query. If the condition is not met, an overhaul is invoked to increase s to the nearest power of 2 at least $i \operatorname{Log} i$. This implies $s \leq 2 \cdot i \log i$. Indeed, if s was increased right before answering the i-th query, then obviously $s \leq 2 \cdot i \log i$, Otherwise, suppose that s was increased to the current value right before answering the i_{last}-th query; then $s \leq 2 \cdot i_{last} \log i_{last} < 2i \log i$.

We can now assert that the i-th query entails a cost of $O(\frac{n}{s} \operatorname{Log} s) = O(\frac{n}{i \operatorname{Log} i} \cdot \log(i \operatorname{Log} i)) = O(n/i)$ time. It thus follows that the total cost of r queries is $O(n \sum_{i=1}^{r} \frac{1}{i}) = O(n \operatorname{Log} r)$. As the final run size s is at most $2r \operatorname{Log} r$, we can conclude that the algorithm processes r queries in $O(n \operatorname{Log} r)$ time.

When $r \operatorname{Log} r \geq n/2$, it holds that $r = \Omega(n/\log n)$. The algorithm can afford to sort the entire S in $O(n \log n) = O(n \log r)$ time and answer every subsequent query in $O(\log n) = O(\log r)$ time. Thus, the algorithm achieves $\textsc{Time}(n, r) = O(n \operatorname{Log} r)$ for all $r \leq n$.

Cracking. This approach mimics the following strategy for building a binary search tree (BST) $\mathcal{T}$ on S: (i) find the median of S, and splits S into S_1 and S_2 at the median; (ii) store the median as the root's key, and then build the root's left (resp., right) subtree recursively on S_1 (resp., S_2). Rather than doing a full construction outright, the algorithm builds $\mathcal{T}$ on an "as-needed" basis during query processing.

In the beginning, only the root exists and it is put in the *unexpanded* mode. In general, an unexpanded node u has no children yet, but is associated with the subset $S_u \subseteq S$ of elements that ought to be stored in its subtree. A query is answered in the same manner as in a normal BST, by traversing a root-to-leaf path π of $\mathcal{T}$. The main difference is that as the search comes to an unexpanded node u on π, the algorithm must *expand* it first. Expanding u means creating two child nodes for u, splitting S_u at the median (the key of u), dividing S_u at the median into two parts, and associating each part with a child. After that, u becomes *expanded* with its children put in the unexpanded mode. The expansion takes $O(|S_u|)$ time (finding the median of a set takes linear time [1]).

After r queries, the BST $\mathcal{T}$ is partially built because only the nodes on the r root-to-leaf paths traversed during query processing are expanded. The nodes at the first $\operatorname{Log} r$ levels[2] can incur a total expansion cost of $O(n \operatorname{Log} r)$. For each root-to-leaf path π, the node u at level $\operatorname{Log} r$ has expansion cost $O(n/r)$, which dominates the total expansion cost of the descendants of u on π. Therefore, other than the nodes at the first $\operatorname{Log} r$ levels, all the other nodes in $\mathcal{T}$ have an expansion cost of $r \cdot O(\frac{n}{r}) = O(n)$ in total. The algorithm therefore achieves $\textsc{Time}(n, r) = O(n \operatorname{Log} r)$ for all $r \leq n$.

[2]The *level* of a node is the number of edges on the path from the node to the root.

4 A Universal Algorithm for $(\log n, \log n)$ Spectrum Indexable Problems

In this section, we will establish:

Theorem 1. *Every* $(\log n, \log n)$ *spectrum indexable problem admits a DDS algorithm that can process any* $r \in [1, n]$ *queries in* $O(n \cdot \operatorname{Log} r)$ *time.*

Next, we prove the theorem by explicitly describing a DDS algorithm achieving $\textsc{Time}(n, r) = O(n \cdot \operatorname{Log} r)$. Assume, w.l.o.g., that n is a power of 2. Define $B(n) = \log n$ and $Q(n) = \log n$.

Our algorithm executes in *epochs*. At the beginning of the i-th ($i \geq 1$) epoch, we set

$$s = 2^{2^i}$$

and create a structure $\mathcal{T}$ — the structure promised by $(B(n), Q(n))$ spectrum indexability — on S in $O(n \cdot B(s))$ time. The structure allows us to answer any query in $O(\frac{n}{s} \cdot Q(s))$ time. The i-th epoch finishes after s queries are answered *during* the epoch. These queries demand a total cost of

$$s \cdot O\Big(\frac{n}{s} \cdot Q(s)\Big) \quad = \quad O(n \cdot Q(s)).$$

It is clear that the total computation time of the i-th epoch is $O(n \cdot B(s) + n \cdot Q(s)) = O(n \cdot 2^i)$, i.e., the time to construct $\mathcal{T}$ dominates the time of answering all the queries in the epoch.

As $n \cdot 2^i$ doubles when i increases by 1, the overall cost of answering r queries is $O(n \cdot 2^{h^*})$, where h^* is the number of epochs needed. Precisely, the value of h^* is the <u>*lowest*</u> integer $h \geq 1$ satisfying two conditions:

- **C1:** $2^{2^h} \leq n$ (the value of s must not exceed n);
- **C2:** $\sum_{i=1}^{h} 2^{2^i} \geq r$ (the number of queries that can be answered by h epochs must be at least r).

Lemma 2. $2^{h^*} = O(\operatorname{Log} r)$.

Proof. The condition **C1** states that $2^h \leq \log n$. It suffices to prove that the lowest h satisfying **C2** fulfills the condition $2^h = O(\operatorname{Log} r)$. This is obvious because (i) for $r < 4$, **C2** is satisfied by $h = 1$; (ii) for $r \geq 4$, **C2** is satisfied by $h = \log \log r$. □

This completes the proof of Theorem 1.

Remark. Theorem 1 is a special case of a result in [3].

References

[1] Manuel Blum, Robert W. Floyd, Vaughan R. Pratt, Ronald L. Rivest, and Robert Endre Tarjan. Time bounds for selection. *Journal of Computer and System Sciences (JCSS)*, 7(4):448–461, 1973.

[2] Richard M. Karp, Rajeev Motwani, and Prabhakar Raghavan. Deferred data structuring. *SIAM Journal on Computing*, 17(5):883–902, 1988.

[3] Yufei Tao. Maximizing the optimality streak of deferred data structuring (a.k.a. database cracking). In *Proceedings of International Conference on Database Theory (ICDT)*, pages 10:1–10:18, 2025.